e0[941854] % 3 == 1 : Go benchmark

Part A: Maximising IPC under Area constraint of 60 units

1. Methodology and Objectives

This analysis focuses on designing an optimal processor architecture for the Go benchmark under strict area constraints. The optimization problem was formulated to maximize Instructions Per Cycle (IPC) and efficiency subject to an area constraint of 60 units. The area calculation models distinguish between in-order processors where Area = W × 1.8 + Int_ALU × 2 + Int_Mult × 3 + FP_ALU × 4 + FP_Mult × 5, and out-of-order processors where Area = (W × 1.8 + Int_ALU × 2 + Int_Mult × 3 + FP_ALU × 4 + FP_Mult × 5) × 1.4 + 0.5 × RUU + 0.5 × LSQ. The performance metrics employed were IPC for part (A) and efficiency defined as IPC divided by watt, where Area = Power, representing instructions per cycle per unit area for part (B).

2. Baseline Configuration Analysis

The **baseline configuration** was established by executing the default SimpleScalar sim-outorder simulator with the command `./../simplesim-3.0/sim-outorder -max:inst 80000000 -dumpconfig default_config.txt go.ss 50 9 2stone9.in > go.out` from the benchmark directory

The default SimpleScalar configuration extracted from the dump revealed a 4-wide out-of-order processor with 4 integer ALUs, 1 integer multiplier, 4 floating-point ALUs, 1 floating-point multiplier, 16-entry reorder buffer, and 8-entry load/store queue. The total area calculation yielded a base area of 39.2 units (4 × 1.8 + 4 × 2 + 1 × 3 + 4 × 4 + 1 × 5) multiplied by the out-of-order factor of 1.4, plus queue contributions of 8 and 4 units respectively, totaling 66.88 units. **This configuration achieved an IPC of 0.9118 cycles per instruction with a CPI of 1.0967, resulting in an efficiency of 0.0136 IPC per unit area**. Critically, the baseline configuration exceeded the area constraint by 11.5%, necessitating optimization to reduce area while maximizing performance.

3. Workload characterization

Workload characterization was performed using instruction class profiling via `./../simplesim-3.0/sim-profile -iclass -max:inst 80000000 go.ss 50 9 2stone9.in > profile_results.out` to understand the computational demands of Go.

The analysis showed load operations at 16,761,025 occurrences (20.95%), store operations at 6,168,970 occurrences (7.71%), unconditional branches at 2,573,668 occurrences (3.22%), conditional branches at 9,260,282 occurrences (11.58%). This demonstrates **memory-intensive behavior with 22,929,995 memory operations representing 28.66% of the 80,000,002 committed instructions, comprising 16,761,025 loads (20.95%) and 6,168,970 stores (7.71%) with a load-to-store ratio of 2.7:1**. Additionally, integer computation dominates with 45,236,041 occurrences (56.55%) compared to floating-point computation with zero occurrences (0.00%), and trap operations with 14 occurrences (0.00%). **With this, the most significant discovery was the complete absence of floating-point computations in the Go benchmark, indicating that the default configuration allocated substantial area (20 units) to entirely unused floating-point resources.**

A CPI breakdown using `timeout 300 ./../simplesim-3.0/sim-outorder -max:inst 80000000 -fetch:ifqsize 4 -decode:width 4 -issue:width 4 -commit:width 4 -res:ialu 4 -res:imult 1 -res:fpalu 4 -res:fpmult 1 -ruu:size 16 -lsq:size 8 go.ss 50 9 2stone9.in > stall_stats.out` decomposed the overall cycles per instruction (CPI) of 1.0967 into base execution (ideal CPI of ~1 for a single-issue pipeline, adjusted for superscalar) plus stall contributions: **IFQ full stalls account for 35.22% of cycles (indicating frequent fetch bottlenecks)**, **RUU full stalls for 19.61% (suggesting instruction window limitations)**, LSQ full stalls for only 1.98% (minimal memory ordering issues), **branch misprediction penalties contributing to ~18.78% directional inaccuracy**, and cache-related stalls from il1 miss rate of 6.19% (high instruction misses) and dl1 miss rate of 1.00% (lower data misses), **highlighting that RUU and fetch/branch inefficiencies dominate performance losses in the Go benchmark**.

4. Search Space Reduction Strategy

Out-Of-Order Reduction in search space for IPC Maximization

Based on the zero floating-point usage discovery, the floating-point functional units were reduced to **minimum allowable values with FP ALUs reduced from 4 to 1 and FP multipliers maintained at the existing minimum value of 1**. This data-driven optimization yielded immediate area savings while maintaining functional correctness and architectural completeness.

Further analysis of the detailed instruction profiling data revealed additional optimization opportunities for integer functional units (Appendix B). The Go benchmark exhibited heavily skewed integer operation usage patterns, with integer addition dominating at 32.63% of total instructions while integer multiplication operations comprised only 0.07% of execution time. **This 489:1 ratio between addition and multiplication frequency (32.63% vs 0.07%) indicated that integer multipliers would experience minimal utilization compared to integer ALUs**. Consequently, the integer multiplier parameter space was constrained from {1,2,3,4} to {1,2}, reflecting the reality that **additional multiplication units beyond two would provide negligible performance benefits** for this workload while consuming valuable area resources for the out-of-order processor.

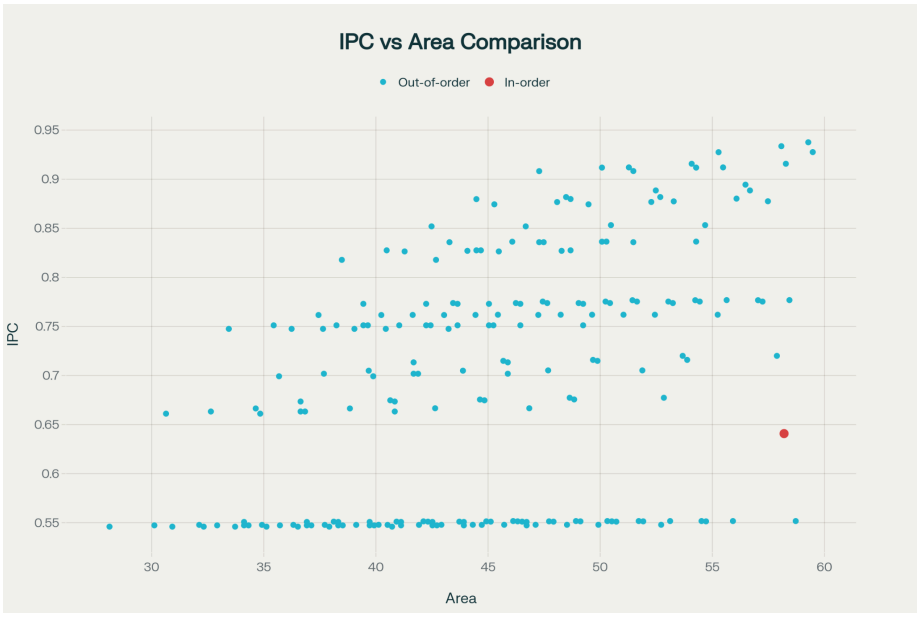
Given the restriction of the integer multiplier functional units to {1,2} from {1,2,3,4} and the restriction of FP multiplier and FP ALU to both {1} unit each, **we now end up with 4 * 2 * 4 * 1 * 1 * 3 * 3 = 288 configs. We experienced simulation saving of (4 ^ 5 * 3 * 3 - 288)/(4 ^ 5 * 3 * 3) * 100% = 96.875% on out of order configs. Out of the total possible out of order configs of 288, only 209 were found to be valid as the other 79 of them either have an area of more than 60 or were not accepted by simscalar (such as pipeline width = 3).**

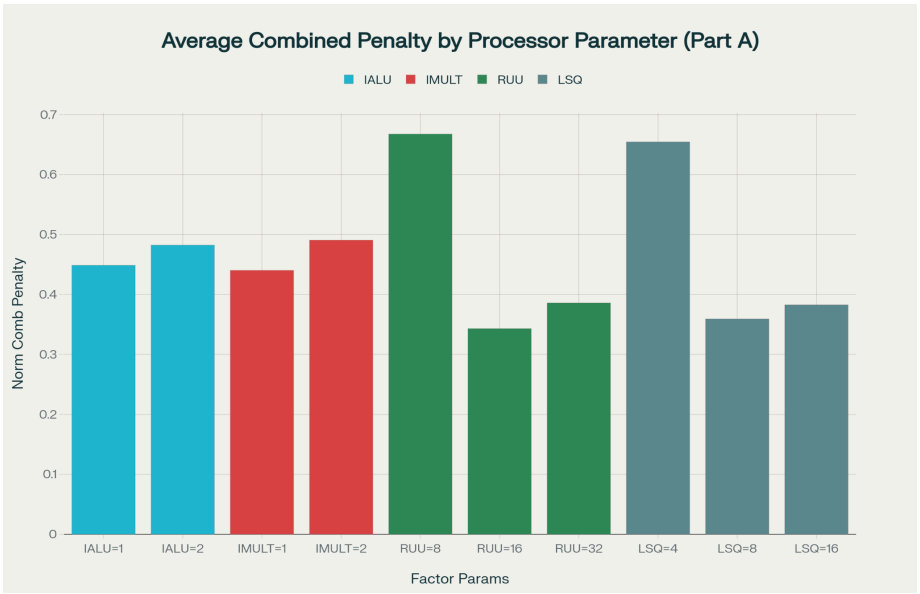
In-Order Execution Elimination for IPC Maximization

To determine whether in-order configurations merited inclusion in the IPC maximization search, a strategic validation test was performed using the maximum feasible in-order configuration within the 60-unit area constraint. The test configuration utilized maximum resources: W=4, IALU=4, IMULT=4, FPALU=4, FPMULT=3, yielding an area of 58.2 units. The simulation command executed was: `timeout 300 ./../simplesim-3.0/sim-outorder -max:inst 80000000 -fetch:ifqsize 4 -decode:width 4 -issue:width 4 -commit:width 4 -issue:inorder true -res:ialu 4 -res:imult 4 -res:fpalu 4 -res:fpmult 3 go.ss 50 9 2stone9.in`.**The results demonstrated that even this maximum resource in-order configuration achieved only IPC = 0.6408,**

substantially lower than the baseline out-of-order performance of IPC = 0.9118. Since this represented the theoretical maximum performance achievable by any in-order configuration within the area budget, **all 1,024 possible in-order configurations (4×4×4×4×4) could be definitively eliminated from the IPC maximization search space, simulation savings on in order configs: 1024/1024 * 100% = 100%**.

5. Analysis of results





Best possible param:

W=4, IALU=3, IMULT=1, FPALU=1, FPMULT=1, RUU=32, LSQ=16, Area=59.28, IPC=0.9376, Efficiency=.015816 (IPC/Power).

Out-of-order configurations (blue) demonstrate superior IPC performance across all area ranges. **The single in-order test point (red) at IPC = 0.6408 represents the theoretical maximum for in-order execution**, confirming that no in-order configuration could compete for IPC leadership. Out-of-order configurations achieve higher IPC even at similar or lower area costs than the maximum in-order configuration.

The bar chart reveals that out-of-order execution structures, particularly the **Register Update Unit (RUU) and Load/Store Queue (LSQ)—dominate processor performance impact, with small RUU and LSQ sizes causing the most severe penalties**; for example, reducing **RUU from 16 to 8 increases the penalty from 0.343 to 0.668, while dropping LSQ from 8 to 4 raises the penalty from 0.359 to 0.655**. Consistent with the CPI breakdown, performance is gated by instruction-windowing (RUU) and memory reordering (LSQ); hence the final OOO design prioritizes larger RUU/LSQ sizes; see *CPI breakdown and Part A results*.

Part B: Maximising efficiency under Area constraint of 60

1. Search Space Reduction Strategy

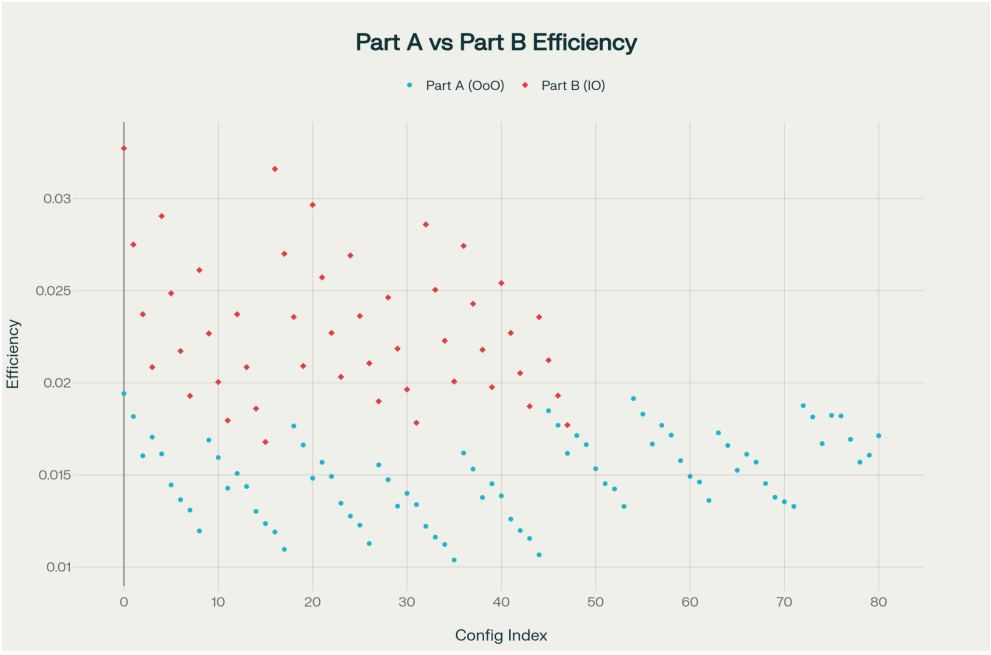
Out-Of-Order Reduction in search space for Efficiency Maximization

The key trick here is to notice that the dataset from part A for the out of order dataset can be reused, given that efficiency was already calculated as part of the config testing script. We can of course apply the same FP functional units reduction technique as before, setting FP ALU to {1} and FP Multiplier to {1}. Since efficiency = IPC/Area (or Power), and we know the FPs are not improving the IPC (the numerator), we can help the denominator of this ratio by removing the functional units, boosting efficiency. The same can be said about the integer multipliers, we can again restrict it to {1,2}, since clearly they save more space than add to IPC (meaning the efficiency ratio of the integer multiplier itself is very poor for this benchmark, contributing to the inefficiency of the entire core). **Config saving is 100% since we are reusing data.**

In-Order Execution Reduction in search space for Efficiency Maximization

The same FP 1/1 constraint is applied to in-order for efficiency runs; full rationale appears in Part A (Search Space Reduction). Once again, having FP units causes inefficiency as they simply take up extra space. Unfortunately, no further search space pruning can be done for this part other than this FP optimization. **Config savings is (1024 - 4^3)/1024 * 100% = 93.75%. Essentially we only need to run 64 more configs.**

2. Analysis of results



Best possible param:

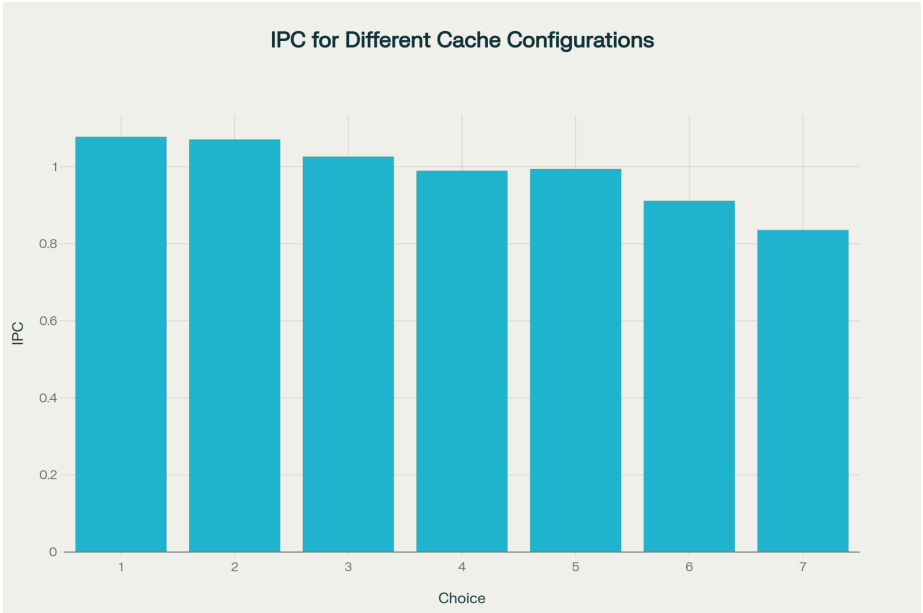
W=1, IALU=1, IMULT=1, FPALU=1, FPMULT=1, inorder, Area=15.8, IPC=0.5171, Efficiency=.032727

The in-order designs (Part B) consistently surpass out-of-order ones (Part A) in area efficiency, forming a tight cluster in the higher band of 0.017–0.033, while out-of-order configs show a greater variability in the lower band of 0.009–0.022. Exploring 48 valid in-order configurations (out of 64 total, with 16 invalid due to exceeding the 60-unit area limit or simulator rejection), the peak efficiency reaches 0.032727 at minimal

resources (W=1, IALU=1, IMULT=1); in contrast, out-of-order designs span 209 valid configurations (out of 288 tested, with 79 invalid) and top out at 0.022353 (W=2, IALU=2, IMULT=1, RUU=8, LSQ=4), underscoring a clear efficiency gap. **This disparity arises because top in-order designs favor narrow pipelines and sparse functional units to minimize area overhead and complexity, whereas out-of-order designs trade efficiency for performance through resource-intensive control structures. Ultimately, a key trade-off emerges: the most efficient in-order design doubles the efficiency of Part A's IPC-optimized out-of-order configuration (0.015816 vs. 0.032727) but at the cost of 44.8% lower IPC (0.5171 vs. 0.9376), though relative to the out-of-order efficiency peak of 0.022353, the gain is approximately 46%.**

Part C: Cache Design

1. Analysis of result



Best possible param:
Choice 1: ICache=1024 sets, DCache=16 sets, IPC=1.0775

Cache test was conducted with the optimal configs from part A. The chart reveals a clear hierarchy in IPC across seven cache configs, with **Choice 1 (il1:1024:64:1:l, dl1:16:64:1:l) achieving the highest IPC of 1.0775, an 29.0% improvement over the worst-performing choice, Choice 7 with an IPC of 0.8356.**

This outcome underscores an inverse relationship between instruction and data cache size: configurations that favor large instruction caches (Choices 1–3, especially with a 1024-set ICache and 16-set DCache) consistently outperform those with more balanced or data-heavy splits, which range from moderate (Choices 4–5, IPC≈0.99) to noticeably degraded (Choices 6–7, IPC<0.92). The performance trend highlights that, for the Go benchmark, **instruction fetch bandwidth and hit rate are far more critical to pipeline efficiency than data cache size**, **indicating an instruction-intensive workload with complex control flow but predictable, localized data access patterns**. These findings suggest that the Go benchmark's architecture benefits **significantly from asymmetric cache hierarchies that heavily prioritize instruction cache capacity while minimizing data cache overhead**.

Conclusion

Under a 60-unit budget, profiling-guided pruning and larger RUU/LSQ delivered the top OOO IPC (0.9376 at area 59.28), cache co-design raised it to 1.0775, and a minimal in-order core maximized IPC/area (0.032727), quantifying the IPC-vs-efficiency trade-off across designs.

Appendix A (Initial workload analysis)

Initial workload analysis (relevant parts)

```
./../simplesim-3.0/sim-outorder -max:inst 80000000 -dumpconfig
default_config.txt go.ss 50 9 2stone9.in > go.out
```

sim_num_insn 80000002 # total number of instructions committed

sim_num_refs 22929995 # total number of loads and stores committed

sim_num_loads 16761025 # total number of loads committed

sim_num_stores 6168970.0000 # total number of stores committed

sim_num_branches 11833950 # total number of branches committed

sim_total_insn 98307304 # total number of instructions executed

sim_total_refs 27877059 # total number of loads and stores executed

sim_total_loads 20868602 # total number of loads executed

sim_total_stores 7008457.0000 # total number of stores executed

sim_total_branches 14434768 # total number of branches executed

sim_IPC 0.9118 # instructions per cycle

sim_CPI 1.0967 # cycles per instruction

bpred_bimod.lookups 15381923 # total number of bpred lookups

bpred_bimod.updates 11833950 # total number of updates

bpred_bimod.addr_hits 9501746 # total number of address-predicted hits

bpred_bimod.dir_hits 9610967 # total number of direction-predicted hits (includes addr-hits)

bpred_bimod.misses 2222983 # total number of misses

bpred_bimod.jr_hits 858529 # total number of address-predicted hits for JR's

bpred_bimod.jr_seen 891580 # total number of JR's seen

bpred_bimod.jr_non_ras_hits.PP 16623 # total number of address-predicted hits for non-RAS JR's

bpred_bimod.jr_non_ras_seen.PP 29309 # total number of non-RAS JR's seen

bpred_bimod.bpred_addr_rate 0.8029 # branch address-prediction rate (i.e., addr-hits/updates)

bpred_bimod.bpred_dir_rate 0.8122 # branch direction-prediction rate (i.e., all-hits/updates)

bpred_bimod.bpred_jr_rate 0.9629 # JR address-prediction rate (i.e., JR addr-hits/JRs seen)

Appendix B (Instruction type breakdown)

Instruction type breakdown

```
./../simplesim-3.0/sim-profile -iprof -max:inst 80000000 go.ss 50 9 2stone9.in
> go_profile.out
```

Integer Addition Operations:

- `addu d,s,t 14071446 17.59`
- `addiu t,s,i 12028290 15.04`
- `add d,s,t 0 0.00`
- `addi t,s,i 0 0.00`

Integer Subtraction Operations:

- `subu d,s,t 398139 0.50`
- `sub d,s,t 0 0.00`

Integer Multiplication Operations:

- `mult s,t 53358 0.07`
- `multu s,t 0 0.00`
- `mflo d 54066 0.07` (move from LO register after multiply/divide)

Integer Division Operations:

- `div s,t 515 0.00`
- `divu s,t 193 0.00`
- `mfhi d 1395 0.00` (move from HI register after divide)

Summary Analysis:

- Integer Addition: 26,099,736 instructions (32.63% of total)
- Integer Subtraction: 398,139 instructions (0.50% of total)
- Integer Multiplication: 53,358 instructions (0.07% of total)
- Integer Division: 708 instructions (0.00% of total)

Appendix C (Data collection)

Part A:

https://drive.google.com/file/d/1w68UA-1DqyiZwIcsjOP0PRdkxrB-XQd2/view?usp=sharing

Part B:
https://drive.google.com/file/d/1LI4tSEE9Cq_2aHUYBNXSQrCBC1uvftB1/view?usp=sharing

Part C

https://drive.google.com/file/d/1iBcYjbtWJ9N3d4Q-gAm7qkLKP2ZPsgyZ/view?usp=sharing